

2.5 二维信号的卷积公式

如 1.2 节所述，在实际应用中，我们碰到的信号不只是一维，还有二维、三维和多维的情况。在这一节，我们将一维信号的卷积公式推广到二维信号中。因为二维信号卷积公式的基本原理与一维信号完全一样，所以我们的推导相对简明扼要一些。由于图像是二维信号，这一节中的内容也可以方便的应用于数字图像处理中。

2.5.1 二维离散信号卷积公式的推导

定义 2.3： 二维离散信号的单位脉冲序列

$$\delta[m, n] = \begin{cases} 1, & \text{当 } m = n = 0 \text{ 时} \\ 0, & \text{其他} \end{cases} \quad (\text{公式 2.41})$$

定义二维离散信号的单位脉冲响应 $h[m, n]$ 为一个 LTI 系统输入是 $\delta[m, n]$ 时的输出：

$$\delta[m, n] \xrightarrow{LTI} h[m, n] \quad (\text{公式 2.42})$$

我们可以将任意一个二维离散信号 $x[m, n]$ 拆解为单位脉冲序列移位和叠加后的信号：

$$x[m, n] = \sum_{k=-\infty}^{+\infty} \sum_{l=-\infty}^{+\infty} x[k, l] \delta[m - k, n - l] \quad (\text{公式 2.43})$$

二维离散信号的卷积公式推导如下：

$$\delta[m, n] \xrightarrow{LTI} h[m, n]$$

$$\delta[m - k, n - l] \xrightarrow{LTI} h[m - k, n - l] \quad (\text{时不变性})$$

$$x[k, l] \delta[m - k, n - l] \xrightarrow{LTI} x[k, l] h[m - k, n - l] \quad (\text{齐次性})$$

$$x[m, n] = \sum_{k=-\infty}^{+\infty} \sum_{l=-\infty}^{+\infty} x[k, l] \delta[m - k, n - l] \xrightarrow{LTI} \sum_{k=-\infty}^{+\infty} \sum_{l=-\infty}^{+\infty} x[k, l] h[m - k, n - l] \quad (\text{叠加性})$$

因此，二维信号的卷积公式为：

$$y[m, n] = x[m, n] * h[m, n] = \sum_{k=-\infty}^{+\infty} \sum_{l=-\infty}^{+\infty} x[k, l] h[m - k, n - l] \quad (\text{公式 2.44})$$

[例 2.34] 计算如下两个二维序列的卷积

$$x[m, n] = \delta[m, n] + 2\delta[m - 1, n] + 3\delta[m - 2, n] + 4\delta[m, n - 1] + 5\delta[m - 1, n - 1] \\ + 6\delta[m - 2, n - 1] + 7\delta[m, n - 2] + 8\delta[m - 1, n - 2] + 9\delta[m - 2, n - 2]$$

$$h[m, n] = \delta[m, n] + 2\delta[m - 1, n] + 3\delta[m, n - 1] + 4\delta[m - 1, n - 1]$$

求 $y[m, n] = x[m, n] * h[m, n]$ 。

解： 将二维卷积公式变形得：

$$\begin{aligned}
 x[m,n] * h[m,n] &= \sum_{k=-\infty}^{+\infty} \sum_{l=-\infty}^{+\infty} x[k,l] h[m-k,n-l] \\
 &= \sum_{k=-\infty}^{+\infty} \sum_{l=-\infty}^{+\infty} x[k,l] h[-(k-m), -(l-n)]
 \end{aligned}$$

求和
相乘
翻转
平移

上述公式表明，我们仍然可以遵循翻转、平移、相乘、求和四步来计算二维信号的卷积。我们首先将 $x[m,n]$ 和 $h[m,n]$ 分别作图，如图 2.54 所示。

		m					m					m				
			0	1	2				0	1			0	1	2	3
	0	1 2 3					0	1 2				0	1 4 7 6			
	1	4 5 6					1	3 4				1	7 23 33 24			
	2	7 8 9										2	19 53 63 42			
												3	21 52 59 36			
		$x[m,n]$					$h[m,n]$					$y[m,n]$				

$x[m,n] \quad * \quad h[m,n] \quad = \quad y[m,n]$

图 2.54 二维卷积示意图

步骤 1: 确定 $y[m,n]$ 取值范围。仍然遵循“**y的最左边（最右边）下标=x的最左边（最右边）下标+h的最左边（最右边）下标**”的原则，不过需要分别对两个维度 m 和 n 来做。在维度 m 上， x 最左边为0，最右边为2；而 h 最左边为0，最右边为1，因此 y 在维度 m 上最左边为0，最右边为3。在维度 n 上， x 最左边为0，最右边为2；而 h 最左边为0，最右边为1，因此 y 在维度 n 上最左边为0，最右边为3。图 2.54 给出了 $y[m,n]$ 的取值范围。

步骤 2: $h[k,l]$ 翻转，得到 $h[-k,-l]$ 。注意这里我们可以先翻转第一个维度，得到 $h[-k,l]$ ，再翻转第二个维度，得到 $h[-k,-l]$ 。当然，我们也可以先翻转第二个维度，得到 $h[k,-l]$ ，再翻转第一个维度，得到 $h[-k,-l]$ 。图 2.55 给出了两种翻转顺序的结果，它们最终的结果是一样的。

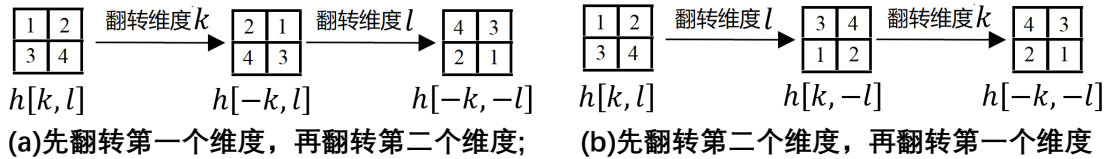


图 2.55 二维卷积中 $h[k,l]$ 的两种翻转方式

步骤 3: 接着是将翻转后的 h 从左到右、从上到下依次平移，与 x 重合，将重合部分相乘并相加，依次得到 $y[m,n]$ 的每一个值。图 2.56 给出了 $y[m,n]$ 的计算示例，请仔细看这张图，对照公式 2.44 深入理解二维卷积的计算过程。最终整理所有计算结果，获得 $y[m,n]$ 如图 2.54 右边所示。

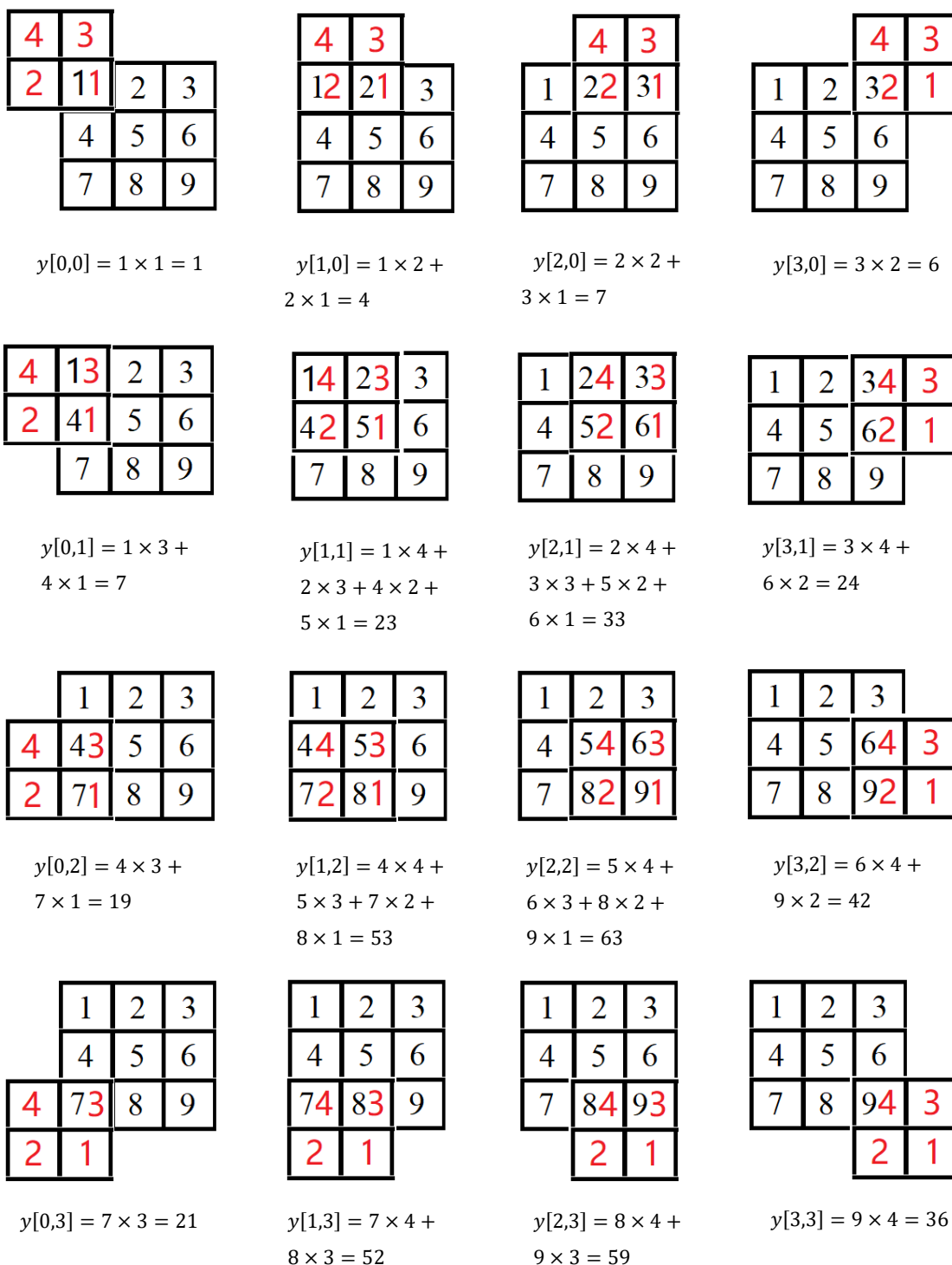


图 2.56 二维卷积计算示例。我们用黑色表示 x 的值，用红色表示 h 的值，将重合的部分相乘并相加，获得 y 相应位置的值。

2.5.2 二维连续信号卷积公式的推导

定义 2.5: 二维冲激信号 $\delta(t_1, t_2)$ 表示为

$$\delta(t_1, t_2) = \begin{cases} +\infty, & \text{当 } t_1 = t_2 = 0 \text{ 时} \\ 0, & \text{其他} \end{cases}, \text{ 且有 } \int_{-\infty}^{+\infty} \int_{-\infty}^{+\infty} \delta(t_1, t_2) dt_1 dt_2 = 1$$

利用勒贝格函数相等概念，可以对冲激信号 $\delta(t_1, t_2)$ 进行更精确的定义。如果一个二维函数 $f(t_1, t_2)$ 满足：对任意“正常”的函数 $y(t_1, t_2)$ 都有：

$$\int_{-\infty}^{+\infty} \int_{-\infty}^{+\infty} y(t_1, t_2) \delta(t_1, t_2) dt_1 dt_2 = y(0, 0)$$

则有：

$$f(t_1, t_2) = \delta(t_1, t_2)$$

假设一个 LTI 系统，当输入冲激函数 $\delta(t_1, t_2)$ 时，输出的冲激响应为 $h(t_1, t_2)$ ，即：

$$\delta(t_1, t_2) \xrightarrow{LTI} h(t_1, t_2)$$

那么这个 LTI 系统对任意一个输入信号 $x(t_1, t_2)$ ，其输出 $y(t_1, t_2)$ 为：

$$y(t_1, t_2) = x(t_1, t_2) * h(t_1, t_2) = \int_{-\infty}^{+\infty} \int_{-\infty}^{+\infty} x(\tau_1, \tau_2) h(t_1 - \tau_1, t_2 - \tau_2) d\tau_1 d\tau_2 \quad (\text{公式 2.44})$$

这就是二维连续信号的卷积公式，其推导与一维情况类似，在这里不详细叙述，请读者自行补充推导细节。

[例 2.35] 若 $x(t_1, t_2) = e^{-a(t_1+t_2)}u(t_1)u(t_2)$ ， $h(t_1, t_2) = e^{-b(t_1+t_2)}u(t_1)u(t_2)$ ，试计算 $y(t_1, t_2) = x(t_1, t_2) * h(t_1, t_2)$

解：直接利用二维卷积公式得：

$$\begin{aligned} y(t_1, t_2) &= \int_{-\infty}^{+\infty} \int_{-\infty}^{+\infty} x(\tau_1, \tau_2) h(t_1 - \tau_1, t_2 - \tau_2) d\tau_1 d\tau_2 \\ &= \int_{-\infty}^{+\infty} \int_{-\infty}^{+\infty} e^{-a(\tau_1+\tau_2)}u(\tau_1)u(\tau_2) e^{-b(t_1-\tau_1+t_2-\tau_2)}u(t_1-\tau_1)u(t_2-\tau_2) d\tau_1 d\tau_2 \end{aligned}$$

若 $t_1 < 0$ 或 $t_2 < 0$ 则上式恒为 0。当且仅当 $t_1 > 0$ 且 $t_2 > 0$ 时，

$$y(t_1, t_2) = e^{-b(t_1+t_2)} \int_0^{t_1} e^{(b-a)\tau_1} d\tau_1 \int_0^{t_2} e^{(b-a)\tau_2} d\tau_2$$

上式分两种情况讨论。

情况 1：当 $a = b$ 时，

$$y(t_1, t_2) = e^{-b(t_1+t_2)} \int_0^{t_1} 1 d\tau_1 \int_0^{t_2} 1 d\tau_2 = t_1 t_2 e^{-b(t_1+t_2)}$$

情况 2：当 $a \neq b$ 时，

$$y(t_1, t_2) = \frac{1}{(b-a)^2} [e^{-a(t_1+t_2)} + e^{-b(t_1+t_2)} - e^{-(at_1+bt_2)} - e^{-(at_2+bt_1)}]$$

综合上述所有情况，获得最终结果如下：

$$y(t_1, t_2) = \begin{cases} t_1 t_2 e^{-b(t_1+t_2)} u(t_1) u(t_2), & \text{当 } a = b \text{ 时} \\ \frac{1}{(b-a)^2} [e^{-a(t_1+t_2)} + e^{-b(t_1+t_2)} - e^{-(at_1+bt_2)} - e^{-(at_2+bt_1)}] u(t_1) u(t_2), & \text{当 } a \neq b \text{ 时} \end{cases}$$

图 2.57 画出了例 2.25 中当 $a = 1$ ， $b = 2$ 时的 $x(t_1, t_2)$ 、 $h(t_1, t_2)$ 和 $y(t_1, t_2)$ 的图像。请参考程序来学习如何利用 MATLAB 画二维函数图。

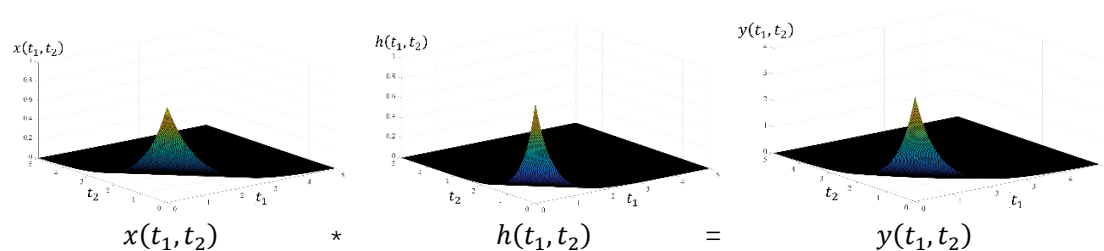


图 2.57 例 2.25 中当 $a = 1$, $b = 2$ 时画出的 $x(t_1, t_2)$ 、 $h(t_1, t_2)$ 和 $y(t_1, t_2)$ 的图像。

图 2.57 的 MATLAB 程序如下：

```
a = 1;
b = 2;
[t1,t2] = meshgrid(0:0.01:5,0:0.01:5);
figure(1)
x = exp(-a*t1-a*t2);
surf(t1,t2,x);
figure(2)
h = exp(-b*t1-b*t2);
surf(t1,t2,h);
figure(3)
y = exp(-a*t1-a*t2)+exp(-b*t1-b*t2)+exp(-a*t1-b*t2)+exp(-b*t1-a*t2);
surf(t1,t2,y)
```

2.6 卷积运算的 MATLAB 编程

2.6.1 离散时间信号卷积的 MATLAB 编程

[MATLAB 编程示例 2.1] 请编写一个函数，完成与 MATLAB 自带的求离散信号的卷积函数 conv 一样的功能。

说明：在 MATLAB 中自带了求离散序列的卷积函数 conv。以下是用法示例。

展示 conv 函数用法的 MATLAB 程序：

```
x = [1,2,3];
h=[2,3,4];
y=conv(x,h)
输出结果：
y = [2,7,16,17,12]
```

对程序的说明：假设有两个离散序列 $x[n]$, $h[n]$ 。不失一般性，我们假设 $x[n]$, $h[n]$ 的非零信号都是从下标 0 开始的（这可以通过对信号的平移做到），这样我们只需要关心两个序列的值，而不用关心它们所在的具体位置。设 $x[n]$ 和 $h[n]$ 的长度分别为 N_1 和 N_2 ，那么 MATLAB 函数 conv 所做的事情就是将两个序列卷积，生成一个长度为 $N_1 + N_2 - 1$ 的序列 $y[n]$ 。在上面例子中， $N_1 = N_2 = 3$ ，因此生成的 $y[n]$ 长度为 7，其具体的值就是 $x[n]$ 和 $h[n]$ 卷积的结果，请读者自行验证结果的正确性。

以下是基于列表法编写的两个离散序列的卷积函数，我们把此函数起名为 conv1D。我

们先回顾一下使用列表法计算离散信号的卷积，我们只需要将信号 $h[n]$ 分别平移 0 到 $N_1 - 1$ 个单位，并乘以 $x[n]$ 对应的数值，然后叠加起来即可。

基于列表法编写的两个离散序列卷积函数 conv1D：

```
function C = conv1D(A,B)
% This is the function to do convolution of two arrays.
C = zeros(1,length(A) + length(B)-1);
for i = 1:length(A)
    C = C + [zeros(1,i-1),A(i)*B, zeros(1,length(A)-i)];
end
```

以上这段代码的意思是，先计算出 A 和 B 卷积后的信号 C 的长度 $\text{length}(A) + \text{length}(B) - 1$ ，然后使用列表法，每次将信号 B 平移 i 个单位，并乘以 $A(i)$ ，最后叠加上去。注意 MATLAB 中向量的下标是从1开始计算的。同时，请注意体会 MATLAB 中数组合并和叠加操作。将以上代码进行测试，可以获得与 MATLAB 中自带的 conv 函数一样的效果。

调用 conv1D 的 MATLAB 程序：

```
x = [1,2,3];
h=[2,3,4];
y=conv1D(x,h)
```

输出结果：

```
y = [2,7,16,17,12]
```

下面这段程序，用来比较 MATLAB 自带的 conv 函数和我们基于列表法编写的 conv1D 函数的计算时间复杂度。这里我们将 x 和 h 的长度都设置为 10000，然后分别调用 conv 和 conv1D 计算卷积，并用 MATLAB 的 tic 和 toc 计算函数运行时间。

比较 MATLAB 自带的 conv 和我们编写的 conv1D 的计算时间：

```
x = rand(1,10000);
h = rand(1,10000);
tic
y = conv(x,h);
toc
```

```
tic
y = conv1D(x,h);
toc
```

输出结果：

历时 0.001633 秒。

历时 0.240119 秒。

可见，MATLAB 自带的 conv 函数处理两个长度 10000 的序列卷积用时 0.001633 秒，而我们编写的 conv1D 处理用时为 0.240119 秒，慢了将近 150 倍。其原因在于，MATLAB 自带的 conv 函数用到了第四章的快速傅里叶变换(FFT)，其时间复杂度为 $\mathcal{O}(N \log N)$ ，而我们基于列表法编写的 conv1D 函数时间复杂度为 $\mathcal{O}(N^2)$ 。在第四章学习了快速傅里叶变换后，我们会重新编写离散序列卷积程序，将会比 conv1D 快很多。

2.6.3 二维离散时间信号卷积的 MATLAB 编程

[MATLAB 编程示例 2.2] 请编写一个函数，完成与 MATLAB 自带的求二维离散信号的卷积函数 conv2 一样的功能。

二维离散卷积和一维离散卷积类似，设两个二维离散信号 $x(u, v)$ ， $h(u, v)$ 的非零值都从 $(0, 0)$ 处开始，且宽度范围为 (N_1, M_1) 和 (N_2, M_2) 。因此卷积输出信号 $y(u, v)$ 的范围是 $(N_1 + N_2 - 1, M_1 + M_2 - 1)$ 。

我们依旧可以使用类似列表法的方法来求解，将信号 $h(u, v)$ 平移 (i, j) 单位，得到 $h(u - i, v - j)$ ，再乘以对应的幅度 $x(i, j)$ 。对 (i, j) 从 $(0, 0)$ 到 $(N_1 - 1, M_1 - 1)$ 遍历，然后依次叠加起来就可以了，图 2.58 给出了二维离散卷积的计算示意图。

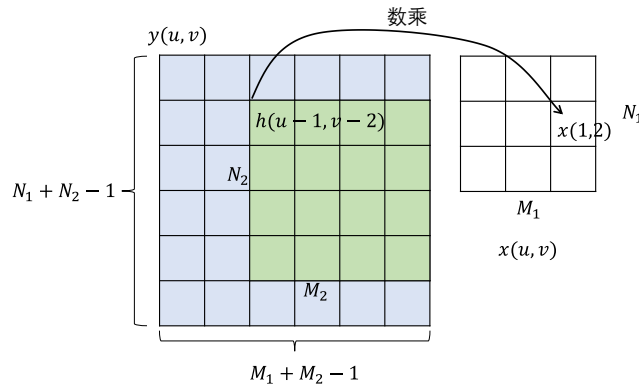


图 2.58 二维卷积的计算示意图

当我们固定住行数为 i ，将 $h(u, v)$ 按行扫过（遍历 j ）时， $h(u, v)$ 的每一行都将平移 j 并和信号 $x(u, v)$ 第 i 行的第 j 个元素相乘并叠加起来，这正是我们在一维卷积中的做法。这意味着当固定住行数为 i 时，输出信号 $y(u, v)$ 的第 i 行到第 $i + N_2 - 1$ 行的最终信号有一个叠加的分量来自于 $h(u, v)$ 每一行和信号 $x(u, v)$ 第 i 行按行卷积的结果。既然如此，我们可以直接使用一维卷积，去计算最终输出 $y(u, v)$ 每一行包含的若干卷积分量。接下来我们只需要知道最终输出 $y(u, v)$ 每一行包含 $h(u, v)$ 和 $x(u, v)$ 哪些行的卷积分量就行了。

我们首先观察 $y(u, v)$ 的第 1 行。显然只有当 $i = 0$ 的时候， $y(u, v)$ 的第 1 行才有被计算到，它只包含 $h(u, v)$ 第 1 行和 $x(u, v)$ 第 1 行的卷积结果。同样的我们也很容易观察到 $y(u, v)$ 的最后一行，即当 $i = N_1 - 1$ 时，此时 $y(u, v)$ 的最后一行只包含 $h(u, v)$ 最后一行和 $x(u, v)$ 最后一行的卷积结果。

根据这样的观察，我们可以发现， $y(u, v)$ 的第 k 行，包含多少个卷积分量，与 i 在遍历的过程中， k 落在区间 $[i, i + N_2 - 1]$ 的次数一样多。

因此，对于第 k 行，要满足 $i \leq k \leq i + N_2 - 1$ ， i 的取值范围为 $k - N_2 + 1 \leq i \leq k$ ，我们还有 $0 \leq i \leq N_1 - 1$ 的约束，因此 i 的取值范围为

$$\max\{k - N_2 + 1, 0\} \leq i \leq \min\{k, N_1 - 1\}$$

此时，对于每一个 i ， $y(u, v)$ 的第 k 行都会包含一个 $x(u, v)$ 的第 i 行和 $h(u, v)$ 的第 $k - i$ 行的卷积结果。

理解到这里，再看下面这段代码就能够理解了，注意 MATLAB 中下标是从 1 开始的。代码中的外层循环 i 其实就是最终输出结果 C 的第 i 行，内层循环的 j 就是去遍历 C 的第 i 行应该包含哪些卷积分量。

计算二维离散序列卷积的 MATLAB 程序：

```

function C = conv2D(A,B)
C = zeros(size(A,1)+size(B,1)-1,size(A,2)+size(B,2)-1);
for i = 1:size(C,1)
    minJ = max(1,i-size(A,1)+1);
    maxJ = min(i,size(B,1));
    for j = minJ:maxJ
        C(i,:) = C(i,:) + conv1D(A(i-j+1,:),B(j,:));
    end
end
end

```

调用 MATLAB 自带的 conv2 和我们编写的 conv2D 的程序：

```

x = [1,2,3;4,5,6;7,8,9];
h=[1,2;3,4];
y=conv2(x,h)
y=conv2D(x,h)

```

输出结果：

```

y =  1     4     7     6
      7    23    33    24
     19    53    63    42
     21    52    59    36
y =  1     4     7     6
      7    23    33    24
     19    53    63    42
     21    52    59    36

```

可见，以上我们编写计算二维离散序列卷积的 conv2D 函数，与 MATLAB 自带的 conv2 输出完全一样。